

Module 2: Essentials of Python

Author: Lei Wu

Date: 2023-01-01

Version: 1.0.0

Learning Objectives

- Advanced Python data structures
- Object-oriented programming concepts
- File handling and I/O operations
- Exception handling
- List comprehensions and advanced features

Topics Covered

1. Lists, tuples, and dictionaries
2. Sets and their operations
3. String manipulation and formatting
4. List comprehensions and generator expressions
5. Object-oriented programming basics
6. Classes and objects
7. Inheritance and polymorphism
8. File reading and writing
9. Exception handling (try/except/finally)
10. Regular expressions
11. Working with dates and time

Exercises

- Hands-on coding exercises for each topic

1. Lists, Tuples, and Dictionaries

```
In [1]: # Lists - mutable ordered collection
        fruits = ["apple", "banana", "orange"]
        print(f"Original list: {fruits}")

        # List operations
        fruits.append("grape")
        print(f"After append: {fruits}")

        fruits.insert(1, "mango")
        print(f"After insert: {fruits}")
```

```

fruits.remove("banana")
print(f"After remove: {fruits}")

# List slicing
print(f"First two items: {fruits[:2]}")
print(f>Last two items: {fruits[-2:]}")

# Tuples – immutable ordered collection
coordinates = (10, 20)
print(f"Coordinates: {coordinates}")
print(f"X coordinate: {coordinates[0]}")

# Tuple unpacking
x, y = coordinates
print(f"X: {x}, Y: {y}")

# Dictionaries – key-value pairs
person = {
    "name": "Alice",
    "age": 30,
    "city": "New York"
}
print(f"Person: {person}")
print(f"Name: {person['name']}")
print(f"Age: {person.get('age', 'Unknown')}")

# Dictionary operations
person["occupation"] = "Engineer"
print(f"After adding occupation: {person}")

# Iterating through dictionary
for key, value in person.items():
    print(f"{key}: {value}")

```

```

Original list: ['apple', 'banana', 'orange']
After append: ['apple', 'banana', 'orange', 'grape']
After insert: ['apple', 'mango', 'banana', 'orange', 'grape']
After remove: ['apple', 'mango', 'orange', 'grape']
First two items: ['apple', 'mango']
Last two items: ['orange', 'grape']
Coordinates: (10, 20)
X coordinate: 10
X: 10, Y: 20
Person: {'name': 'Alice', 'age': 30, 'city': 'New York'}
Name: Alice
Age: 30
After adding occupation: {'name': 'Alice', 'age': 30, 'city': 'New York',
'occupation': 'Engineer'}
name: Alice
age: 30
city: New York
occupation: Engineer

```

2. Sets and Their Operations

```

In [2]: # Creating sets
fruits = {"apple", "banana", "orange", "apple"} # Duplicates removed
print(f"Fruits set: {fruits}")

```

```

numbers = set([1, 2, 3, 4, 5, 2, 3]) # From list
print(f"Numbers set: {numbers}")

# Set operations
set1 = {1, 2, 3, 4, 5}
set2 = {4, 5, 6, 7, 8}

print(f"Set 1: {set1}")
print(f"Set 2: {set2}")

# Union
union = set1 | set2
print(f"Union: {union}")

# Intersection
intersection = set1 & set2
print(f"Intersection: {intersection}")

# Difference
difference = set1 - set2
print(f"Set1 - Set2: {difference}")

# Symmetric difference
symmetric_diff = set1 ^ set2
print(f"Symmetric difference: {symmetric_diff}")

# Set methods
set1.add(6)
print(f"After adding 6: {set1}")

set1.remove(1)
print(f"After removing 1: {set1}")

# Check membership
print(f"Is 3 in set1? {3 in set1}")
print(f"Is 10 in set1? {10 in set1}")

```

```

Fruits set: {'apple', 'orange', 'banana'}
Numbers set: {1, 2, 3, 4, 5}
Set 1: {1, 2, 3, 4, 5}
Set 2: {4, 5, 6, 7, 8}
Union: {1, 2, 3, 4, 5, 6, 7, 8}
Intersection: {4, 5}
Set1 - Set2: {1, 2, 3}
Symmetric difference: {1, 2, 3, 6, 7, 8}
After adding 6: {1, 2, 3, 4, 5, 6}
After removing 1: {2, 3, 4, 5, 6}
Is 3 in set1? True
Is 10 in set1? False

```

3. String Manipulation and Formatting

```

In [3]: # Basic string operations
text = "Hello, World!"
print(f"Original: {text}")
print(f"Uppercase: {text.upper()}")
print(f"Lowercase: {text.lower()}")
print(f"Title case: {text.title()}")
print(f"Length: {len(text)}")

```

```

# String methods
sentence = " Python is awesome! "
print(f"Original: '{sentence}'")
print(f"Stripped: '{sentence.strip()}'")
print(f"Left stripped: '{sentence.lstrip()}'")
print(f"Right stripped: '{sentence.rstrip()}'")

# String splitting and joining
words = "apple,banana,orange,grape"
fruit_list = words.split(",")
print(f"Split by comma: {fruit_list}")

joined = " | ".join(fruit_list)
print(f"Joined with ' | ': {joined}")

# String formatting methods
name = "Alice"
age = 30

# f-string (Python 3.6+)
message1 = f"Hello, {name}! You are {age} years old."
print(f"f-string: {message1}")

# .format() method
message2 = "Hello, {}! You are {} years old.".format(name, age)
print(f".format(): {message2}")

# % formatting
message3 = "Hello, %s! You are %d years old." % (name, age)
print(f"% formatting: {message3}")

# String checking methods
email = "user@example.com"
print(f"Is email alphanumeric? {email.isalnum()}")
print(f"Is email lowercase? {email.islower()}")
print(f"Does email start with 'user'? {email.startswith('user')}")
print(f"Does email end with '.com'? {email.endswith('.com')}")

# String replacement
text = "I love Python programming"
new_text = text.replace("Python", "Java")
print(f"Original: {text}")
print(f"Replaced: {new_text}")

```

Original: Hello, World!
Uppercase: HELLO, WORLD!
Lowercase: hello, world!
Title case: Hello, World!
Length: 13
Original: ' Python is awesome! '
Stripped: 'Python is awesome!'
Left stripped: 'Python is awesome! '
Right stripped: ' Python is awesome!'
Split by comma: ['apple', 'banana', 'orange', 'grape']
Joined with ' | ': apple | banana | orange | grape
f-string: Hello, Alice! You are 30 years old.
.format(): Hello, Alice! You are 30 years old.
% formatting: Hello, Alice! You are 30 years old.
Is email alphanumeric? False
Is email lowercase? True
Does email start with 'user'? True
Does email end with '.com'? True
Original: I love Python programming
Replaced: I love Java programming

4. List Comprehensions and Generator Expressions

```
In [4]: # List comprehensions
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Basic list comprehension
squares = [x**2 for x in numbers]
print(f"Squares: {squares}")

# List comprehension with condition
even_squares = [x**2 for x in numbers if x % 2 == 0]
print(f"Even squares: {even_squares}")

# List comprehension with multiple conditions
filtered = [x for x in numbers if x > 5 and x % 2 == 0]
print(f"Numbers > 5 and even: {filtered}")

# List comprehension with if-else
categorized = ["even" if x % 2 == 0 else "odd" for x in numbers]
print(f"Categorized: {categorized}")

# Nested list comprehension
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flattened = [item for row in matrix for item in row]
print(f"Flattened matrix: {flattened}")

# Dictionary comprehension
squares_dict = {x: x**2 for x in range(1, 6)}
print(f"Squares dictionary: {squares_dict}")

# Set comprehension
unique_lengths = {len(word) for word in ["apple", "banana", "grape", "kiw"]}
print(f"Unique word lengths: {unique_lengths}")

# Generator expressions (memory efficient)
squares_gen = (x**2 for x in numbers)
print(f"Generator: {squares_gen}")
print(f"First 5 squares: {list(squares_gen)[:5]}")
```

```
# Generator with condition
even_squares_gen = (x**2 for x in numbers if x % 2 == 0)
print(f"Even squares generator: {list(even_squares_gen)}")
```

```
Squares: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
Even squares: [4, 16, 36, 64, 100]
Numbers > 5 and even: [6, 8, 10]
Categorized: ['odd', 'even', 'odd', 'even', 'odd', 'even', 'odd', 'even', 'odd', 'even']
Flattened matrix: [1, 2, 3, 4, 5, 6, 7, 8, 9]
Squares dictionary: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
Unique word lengths: {4, 5, 6}
Generator: <generator object <genexpr> at 0x1113d53c0>
First 5 squares: [1, 4, 9, 16, 25]
Even squares generator: [4, 16, 36, 64, 100]
```

5. Object-Oriented Programming Basics

```
In [5]: # Basic class definition
class Person:
    # Class variable
    species = "Homo sapiens"

    def __init__(self, name, age):
        # Instance variables
        self.name = name
        self.age = age

    def greet(self):
        return f"Hello, I'm {self.name} and I'm {self.age} years old."

    def have_birthday(self):
        self.age += 1
        return f"Happy birthday! Now I'm {self.age} years old."

# Creating objects
person1 = Person("Alice", 25)
person2 = Person("Bob", 30)

print(person1.greet())
print(person2.greet())

print(person1.have_birthday())
print(f"Species: {Person.species}")

# Accessing attributes
print(f"Person1 name: {person1.name}")
print(f"Person1 age: {person1.age}")

# Modifying attributes
person1.age = 26
print(f"Updated age: {person1.age}")
```

Hello, I'm Alice and I'm 25 years old.
Hello, I'm Bob and I'm 30 years old.
Happy birthday! Now I'm 26 years old.
Species: Homo sapiens
Person1 name: Alice
Person1 age: 26
Updated age: 26

6. Inheritance and Polymorphism

```
In [6]: # Base class
class Animal:
    def __init__(self, name, species):
        self.name = name
        self.species = species

    def make_sound(self):
        return "Some generic animal sound"

    def move(self):
        return f"{self.name} moves around"

# Derived class (inheritance)
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name, "Canis lupus")
        self.breed = breed

    def make_sound(self): # Method overriding
        return "Woof! Woof!"

    def fetch(self):
        return f"{self.name} fetches the ball"

# Another derived class
class Cat(Animal):
    def __init__(self, name, color):
        super().__init__(name, "Felis catus")
        self.color = color

    def make_sound(self): # Method overriding
        return "Meow! Meow!"

    def climb(self):
        return f"{self.name} climbs the tree"

# Creating objects
dog = Dog("Buddy", "Golden Retriever")
cat = Cat("Whiskers", "Orange")

# Polymorphism in action
animals = [dog, cat]

for animal in animals:
    print(f"{animal.name} ({animal.species}): {animal.make_sound()}")
    print(f"{animal.move()}")
    print()

# Accessing specific methods
```

```
print(dog.fetch())
print(cat.climb())
```

Buddy (Canis lupus): Woof! Woof!
Buddy moves around

Whiskers (Felis catus): Meow! Meow!
Whiskers moves around

Buddy fetches the ball
Whiskers climbs the tree

7. File Reading and Writing

```
In [7]: # Writing to a file
data = ["Hello, World!", "This is a test file.", "Python is awesome!"]

# Write mode
with open("sample.txt", "w") as file:
    for line in data:
        file.write(line + "\n")
print("File written successfully!")

# Reading from a file
with open("sample.txt", "r") as file:
    content = file.read()
    print("File content:")
    print(content)

# Reading line by line
print("\nReading line by line:")
with open("sample.txt", "r") as file:
    for i, line in enumerate(file, 1):
        print(f"Line {i}: {line.strip()}")

# Appending to a file
with open("sample.txt", "a") as file:
    file.write("This line was appended.\n")
print("\nLine appended!")

# Reading the updated file
print("\nUpdated file content:")
with open("sample.txt", "r") as file:
    print(file.read())

# Working with CSV-like data
import csv

# Writing CSV data
csv_data = [
    ["Name", "Age", "City"],
    ["Alice", "25", "New York"],
    ["Bob", "30", "Los Angeles"],
    ["Charlie", "35", "Chicago"]
]

with open("people.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerows(csv_data)
```



```

print("CSV file written!")

# Reading CSV data
print("\nCSV file content:")
with open("people.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)

```

File written successfully!
File content:
Hello, World!
This is a test file.
Python is awesome!

Reading line by line:
Line 1: Hello, World!
Line 2: This is a test file.
Line 3: Python is awesome!

Line appended!

Updated file content:
Hello, World!
This is a test file.
Python is awesome!
This line was appended.

CSV file written!

CSV file content:
['Name', 'Age', 'City']
['Alice', '25', 'New York']
['Bob', '30', 'Los Angeles']
['Charlie', '35', 'Chicago']

8. Exception Handling

```

In [8]: # Basic exception handling
def safe_divide(a, b):
    try:
        result = a / b
        return result
    except ZeroDivisionError:
        print("Error: Cannot divide by zero!")
        return None
    except TypeError:
        print("Error: Invalid input types!")
        return None

print(safe_divide(10, 2))
print(safe_divide(10, 0))
print(safe_divide(10, "2"))

# Multiple exception types
def process_list(lst, index):
    try:
        return lst[index]

```

```

except IndexError:
    print(f"Error: Index {index} is out of range!")
    return None
except TypeError:
    print("Error: Invalid index type!")
    return None
except Exception as e:
    print(f"Unexpected error: {e}")
    return None

numbers = [1, 2, 3, 4, 5]
print(process_list(numbers, 2))
print(process_list(numbers, 10))
print(process_list(numbers, "2"))

# try-except-else-finally
def process_file(filename):
    file = None
    try:
        file = open(filename, 'r')
        content = file.read()
        print("File read successfully!")
    except FileNotFoundError:
        print(f"Error: File '{filename}' not found!")
        return None
    except PermissionError:
        print(f"Error: Permission denied to read '{filename}'")
        return None
    except Exception as e:
        print(f"Unexpected error: {e}")
        return None
    else:
        print("No errors occurred during file reading")
        return content
    finally:
        if file:
            file.close()
            print("File closed successfully")

# This will show the error handling
result = process_file("nonexistent.txt")

# Custom exceptions
class CustomError(Exception):
    def __init__(self, message):
        self.message = message
        super().__init__(self.message)

def validate_age(age):
    if age < 0:
        raise CustomError("Age cannot be negative!")
    if age > 150:
        raise CustomError("Age seems unrealistic!")
    return True

try:
    validate_age(-5)
except CustomError as e:
    print(f"Custom error: {e}")

```

```

try:
    validate_age(25)
    print("Age validation passed!")
except CustomError as e:
    print(f"Custom error: {e}")

```

```

5.0
Error: Cannot divide by zero!
None
Error: Invalid input types!
None
3
Error: Index 10 is out of range!
None
Error: Invalid index type!
None
Error: File 'nonexistent.txt' not found!
Custom error: Age cannot be negative!
Age validation passed!

```

9. Regular Expressions

```

In [9]: import re

# Basic pattern matching
text = "The quick brown fox jumps over the lazy dog"
pattern = r"fox"
match = re.search(pattern, text)
if match:
    print(f"Found '{match.group()}' at position {match.start()}")
else:
    print("Pattern not found")

# Finding all matches
pattern = r"\b\w{4}\b" # 4-letter words
matches = re.findall(pattern, text)
print(f"4-letter words: {matches}")

# Email validation
email_pattern = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"
emails = ["user@example.com", "invalid.email", "test@domain.co.uk", "not-

for email in emails:
    if re.match(email_pattern, email):
        print(f"'{email}' is a valid email")
    else:
        print(f"'{email}' is not a valid email")

# Phone number pattern
phone_pattern = r"((\d{3})?[-.\s]?(\d{3})[-.\s]?(\d{4})"
phone_text = "Call me at (555) 123-4567 or 555.123.4567 or 555-123-4567"
phone_numbers = re.findall(phone_pattern, phone_text)
print(f"Phone numbers found: {phone_numbers}")

# Substitution
text = "The price is $19.99 and $29.99"
new_text = re.sub(r"\$\d+\.\d{2}", "PRICE", text)
print(f"Original: {text}")
print(f"After substitution: {new_text}")

```

```

# Splitting with regex
text = "apple,banana;cherry:date"
fruits = re.split(r"[,:]", text)
print(f"Split fruits: {fruits}")

# Groups and capturing
pattern = r"(\d{4})-(\d{2})-(\d{2})"
date_text = "Today is 2024-01-15"
match = re.search(pattern, date_text)
if match:
    year, month, day = match.groups()
    print(f"Year: {year}, Month: {month}, Day: {day}")

```

Found 'fox' at position 16
 4-letter words: ['over', 'lazy']
 'user@example.com' is a valid email
 'invalid.email' is not a valid email
 'test@domain.co.uk' is a valid email
 'not-an-email' is not a valid email
 Phone numbers found: ['(555) 123-4567', '555.123.4567', '555-123-4567']
 Original: The price is \$19.99 and \$29.99
 After substitution: The price is PRICE and PRICE
 Split fruits: ['apple', 'banana', 'cherry', 'date']
 Year: 2024, Month: 01, Day: 15

10. Working with Dates and Time

```

In [10]: from datetime import datetime, date, time, timedelta
import time as time_module

# Current date and time
now = datetime.now()
today = date.today()
current_time = time(now.hour, now.minute, now.second)

print(f"Current datetime: {now}")
print(f"Today's date: {today}")
print(f"Current time: {current_time}")

# Creating specific dates and times
birthday = date(1990, 5, 15)
appointment = datetime(2024, 3, 15, 14, 30, 0)

print(f"Birthday: {birthday}")
print(f"Appointment: {appointment}")

# Date arithmetic
age = today - birthday
print(f"Age in days: {age.days}")
print(f"Age in years: {age.days // 365}")

# Adding/subtracting time
future_date = today + timedelta(days=30)
past_date = today - timedelta(weeks=2)

print(f"30 days from now: {future_date}")
print(f"2 weeks ago: {past_date}")

```

```

# Formatting dates
formatted_date = now.strftime("%Y-%m-%d %H:%M:%S")
print(f"Formatted date: {formatted_date}")

# Parsing dates from strings
date_string = "2024-01-15 14:30:00"
parsed_date = datetime.strptime(date_string, "%Y-%m-%d %H:%M:%S")
print(f"Parsed date: {parsed_date}")

# Time zones (basic example)
print(f"Current timestamp: {now.timestamp()}")

# Sleep example (commented out to avoid delay)
# print("Sleeping for 2 seconds...")
# time_module.sleep(2)
# print("Awake!")

# Date comparison
date1 = date(2024, 1, 15)
date2 = date(2024, 1, 20)

if date1 < date2:
    print(f"{date1} is before {date2}")
else:
    print(f"{date1} is after {date2}")

# Working with weekdays
weekdays = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Satu
today_weekday = weekdays[today.weekday()]
print(f"Today is {today_weekday}")

# Days until next Monday
days_until_monday = (7 - today.weekday()) % 7
if days_until_monday == 0:
    days_until_monday = 7
print(f"Days until next Monday: {days_until_monday}")

```

Current datetime: 2025-09-07 21:55:56.626558

Today's date: 2025-09-07

Current time: 21:55:56

Birthday: 1990-05-15

Appointment: 2024-03-15 14:30:00

Age in days: 12899

Age in years: 35

30 days from now: 2025-10-07

2 weeks ago: 2025-08-24

Formatted date: 2025-09-07 21:55:56

Parsed date: 2024-01-15 14:30:00

Current timestamp: 1757307356.626558

2024-01-15 is before 2024-01-20

Today is Sunday

Days until next Monday: 1

In []: